



MID-TERM PROJECT · 20% OF THE FINAL MARK

Group Assignment & Presentation Guide

Six groups · Six topics · One C++ demo each · Peer-graded exercises

Data Structures & Algorithms (INT1306_CLC) · Faculty of Information Technology 2, PTIT



HK I - 2026-2027

Proposed by Dr. HieuLN | PTIT HCMC Campus, TP.HCM

Why you present instead of just listening

The goal

Nobody understands a data structure by hearing about it. You understand it when you have to teach it, implement it, defend it under questions, and then judge whether someone else got it right.

- You will own one topic deeply instead of skimming all of them.
- You will write code that has to work in front of the class.
- You will find out what your structure costs — in C++ and in four other languages.
- You will practise technical English: slides, code comments, report and Q&A.
- You will grade your classmates' work, which forces you to define what 'correct' means.

6

groups

1

topic per group

45 min

per presentation

20%

of your final mark

What every group must deliver — six parts

A

Position & purpose

Where your data structures appear in this syllabus, and why they exist at all.

B

Associated algorithms

The algorithms bound to those structures, with complexity for each operation.

C

Real-world C++ demo

A working program that solves an actual problem — not a textbook snippet.

D

Cross-language comparison

The same problem in Python, Java, JavaScript, Go — measured and evaluated.

E

Exercises for the class

Tasks you set for the other five groups, with a published deadline.

F

Peer grading with evidence

You mark those submissions; the marks feed your classmates' attendance score.

TOPIC ALLOCATION

The six topics

Group 1	Linked Lists	§4.1	Singly · doubly · circular · sorted lists
Group 2	Stacks & the Queue family	§4.2 – 4.3	Stack · queue · circular queue · deque · priority queue
Group 3	Sorting techniques	§2.1	Selection · Insertion · Bubble · Quick · Merge · Radix
Group 4	Searching & Hash tables	§2.2	Linear · Binary · Interpolation search · hashing & collisions
Group 5	Trees & Heaps	§4.4	Binary tree · BST · AVL · min/max heap
Group 6	Graphs	§4.5	Representations · DFS/BFS · Dijkstra, Bellman-Ford, Floyd · Kruskal, Prim

Groups are formed in week 1; topics are confirmed by the lecturer. No two groups may take the same topic.

Position & purpose of your data structures

Answer these, explicitly

- Where in the syllabus does each structure appear — chapter, section, page?
- Which learning outcome (LO1 / LO2 / LO3) does it serve?
- What problem does it exist to solve? What was unacceptable before it?
- Which structures does it build on, and which are built on top of it?
- When is it the wrong choice? Name a case where you would not use it.

Worked shape of an answer

"The circular queue appears in §4.3.2. It exists because a linear array queue wastes the front of the array after repeated dequeues. It serves LO3. It is built on the plain array and is the basis of ring buffers in audio and networking. It is the wrong choice when the queue must grow without bound."

Deliverable for Part A

- A map slide: your structures placed on the syllabus outline.
- One paragraph of motivation per structure.
- Cited sources — textbook chapter or documentation URL.

PART B

The algorithms bound to your structures

Required table. For every structure in your topic, tabulate every operation with its best / average / worst complexity, and say why. A number without a justification scores nothing.

Operation	Best	Average	Worst	Space	Why
Search / find	$O(1)$	$O(n)$	$O(n)$	$O(1)$	No index — every node must be visited in the worst case.
Insert at head	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Only two pointer writes; position is already known.
Insert at position k	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Traversal to k dominates the pointer work.
Delete by value	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Requires a search first.
Traverse all	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Each node is visited exactly once.

Example filled in for a singly linked list (Group 1). Build the equivalent table for your own topic.

The real-world C++ demo

Rules

- It must solve a problem a real user would have — not "insert 5 numbers into a list".
- Implement the structure yourself. You may compare against the STL, but not substitute it.
- Must compile with `g++ -std=c++17 -Wall` and run live in class.
- Must handle at least one dataset large enough to make the complexity visible ($n \geq 100\,000$).
- Must print timing so the class can see the cost, not just be told about it.
- Include the failure cases: empty input, duplicate keys, out-of-range access.

Suggested demo problems

G1

Browser history / undo-redo with a doubly linked list

G2

Hospital triage queue or print spooler with a priority queue

G3

Sorting 1M transaction records, comparing six sorts

G4

Autocomplete / phone-book lookup with a hash table

G5

Leaderboard with a heap, or a dictionary index with an AVL tree

G6

Shortest metro route in Hanoi with Dijkstra

You may propose your own problem — get it approved before you start building.

How to measure — the pattern we expect to see

```
#include <chrono>
#include <iostream>
using namespace std;
using namespace std::chrono;

template <typename F>
double timeIt(F work, int repeats = 5) {
    double best = 1e18;
    for (int r = 0; r < repeats; ++r) {
        auto t0 = high_resolution_clock::now();
        work();
        auto t1 = high_resolution_clock::now();
        double ms = duration<double, milli>(t1 -
t0).count();
        best = min(best, ms);
    }
    return best;    // report the best of 5 runs
}
```

Report, for every n

- Input size n
- Operation count (if you instrument it)
- Wall-clock time, best of 5 runs
- Peak memory, if it is relevant

State your test conditions

CPU and RAM · OS · compiler and version ·
optimisation flags · dataset and how it was
generated.

A measurement without its conditions is not a
measurement.

The Demo Script — How to execute your live showcase

A good demo proves your code works step-by-step. Do not just print a final number. Your live presentation must follow this exact sequence:

1. Data Verification (1 min):

Show the generated dataset (e.g., `head data.csv`). Prove to the class that the input is real, randomized, and correctly sized ($n \geq 100,000$).

2. Edge Case Execution (2 mins):

- Run the program with malicious/edge inputs *first*.
- *Demo*: Empty file, duplicate keys, or out-of-bounds access. Show how your structure gracefully handles or rejects them.

3. Core Algorithm Execution & Verification (3 mins):

- Run the main algorithm on a smaller subset first ($n = 50$) and print the output to visually prove correctness to the class.

4. Stress Test & Profiling (6 mins):

- Run the code on the full dataset ($n = 100,000 +$).
- Display the live terminal output showing the structured timing format (from the `timelt` template).

PART D

Compare C++ with Python, Java, JavaScript and Go

Same problem, same input, five languages. Port your demo (or its core loop) and fill this table with your own measurements — not numbers copied from a blog.

Criterion	C++	Python	Java	JavaScript	Go
Built-in structure used	—	—	—	—	—
Manual implementation effort	—	—	—	—	—
Runtime on your dataset	—	—	—	—	—
Memory footprint	—	—	—	—	—
Memory management	—	—	—	—	—
Type safety	—	—	—	—	—
Readability of the code	—	—	—	—	—
Where you would use it	—	—	—	—	—

Fill every cell. The last row — where you would use it — is what earns the marks.

What a good comparison actually says

1

Be fair

Same algorithm, same input, same machine. Comparing your hand-written C++ list against Python's built-in list is not a language comparison — say which you are doing.

2

Explain the gap

"Python was 40× slower" is an observation. "Python was 40× slower because each element is a boxed object reached through a pointer, and the loop runs in the interpreter" is an answer.

3

Cover the trade-offs

Runtime, memory, lines of code, time you spent writing it, how easy the bug was to find. Speed is not the only axis.

4

Name the right tool

End with a recommendation: which language would you actually pick for this problem in production, and why. Defend it in Q&A.

Angles worth covering

- Manual pointers vs. garbage collection
- Static vs. dynamic typing
- Compiled vs. interpreted vs. JIT
- Standard-library coverage of your structure
- Concurrency support, if your topic needs it
- Tooling: debugger, profiler, package manager

PART D

Empirical Evidence — Prove your comparison

Filling out the comparison table is not enough. You must prove your findings using rigorous empirical data visualization and profiling tools.

VISUALIZING RUNTIME COMPLEXITY:

1

Do not use a single data point. Run your code across multiple dataset sizes (e.g., $n = 10k, 50k, 100k, 500k, 1M$).

Deliverable: A Line Chart (plotted in Excel, Python matplotlib, etc.) showing the runtime growth curves of all 5 languages overlaid on one graph.

RIGOROUS MEMORY PROFILING:

2

You cannot guess memory usage. You must use language-specific profiling tools to capture the exact peak memory footprint of your data structure. *Suggested Tools:* Valgrind/Massif (C++), memory profiler or tracemalloc (Python), VisualVM (Java), Chrome DevTools (Node.js/JS), pprof (Go).

Deliverable: A Bar Chart comparing peak memory consumption (in MBs) across the 5 implementations.

STATISTICAL FAIRNESS

3

Run each test at least 5 times (warm-up runs do not count for Java/JS JIT compilers). Report the average and the variance. If Java is slow on run 1 but fast on run 5, explain why (JIT compilation).

Set exercises for the rest of the class

Requirements

- Publish 3–5 exercises immediately after your presentation.
- Cover a range: one recall question, two implementation tasks, one analysis task, one open/design task.
- Every exercise must be answerable from what you taught — no material you did not present.
- Publish a marking scheme with the exercises: total points and what each point is for.
- State the deadline (minimum one week) and the submission channel.
- Prepare your own reference solution before you publish.

Example set — Group 3, Sorting

Recall

State the worst-case complexity of Quick sort and the input that triggers it.

1 pt

Implement

Write Merge sort in C++ and sort the supplied 100k-line file.

3 pts

Implement

Modify Bubble sort to stop early on a sorted array; show the new best case.

2 pts

Analyse

Count the comparisons of Insertion sort on the supplied dataset; compare to $n^2/2$.

2 pts

Design

You must sort 10 GB that does not fit in RAM. Which algorithm, and why?

2 pts

Grade your classmates — this is not a formality

The marks your group awards feed directly into your classmates' attendance and participation score — 10% of their final grade. Treat it with the seriousness that implies.

1 Publish the scheme first

Points per exercise, and per criterion, before anyone submits.

2 Mark independently

At least two members of your group mark every submission separately.

3 Reconcile

Where the two marks differ by more than 20%, discuss and record the agreed mark.

4 Write the feedback

Two or three sentences per submission: what was right, what was wrong, what to fix.

5 Submit the evidence pack

Marks, feedback, and the artefacts you based them on — to the lecturer.

6 Face the appeal

Any student may contest a mark. Your group must justify it with the evidence.

The evidence pack — what "with evidence" means

Every mark must be traceable

- The submitted file or repository link, with its timestamp.
- The marking sheet: one row per student, one column per criterion.
- A screenshot or terminal log showing the submitted code compiling and running.
- The written feedback you sent back.
- Names of the two markers, and the reconciled mark where they disagreed.
- An attendance record for the session your exercises were set in.

Marking sheet — required columns

```
student_id | name | ex1 | ex2 | ex3 | ex4 |  
ex5  
total | marker_A | marker_B | agreed | note  
  
B21DCCN045 | Tran B | 1 | 2.5 | 2 | 2 | 1  
8.5 | Hoa | Minh | 8.5 | "ex2: merge step  
loses duplicates, -0.5"
```

Zero tolerance

Marks awarded without evidence are void — the lecturer re-marks the whole set and the grading component of your group's project score is forfeited. Inflating marks for friends, or deflating them for rivals, is an academic integrity matter.

How your 45 minutes are spent

8 min	Position & purpose	Part A — where your structures live and why
10 min	Algorithms & complexity	Part B — the operation table, defended
12 min	Live C++ demo	Part C — compile and run it in front of the class
8 min	Cross-language comparison	Part D — your measurements and your recommendation
3 min	Hand out the exercises	Part E — publish tasks, scheme and deadline
4 min	Q&A	Anyone may ask; every member must be able to answer

Non-negotiable

- Everyone speaks. A silent member scores zero.
- English throughout, including Q&A.
- No reading from the slides.
- Bring the demo on two machines — no excuses.
- Send slides + code 24 h in advance.

ASSESSMENT

How the lecturer grades your group

Part A — position & purpose		10 pts	Correct placement in the syllabus, real motivation, cited.
Part B — algorithms & complexity		20 pts	Complete operation table, correct bounds, justified.
Part C — C++ demo		25 pts	Compiles, runs live, solves a real problem, measured, handles edge cases.
Part D — language comparison		20 pts	Fair method, own measurements, explained gaps, clear recommendation.
Part E — exercises set		10 pts	Range of difficulty, answerable, published with a marking scheme.
Part F — grading with evidence		15 pts	Complete evidence pack, independent double marking, useful feedback.

Total 100 points → scaled to the 20% mid-term project component. Individual marks may be adjusted up or down by up to 20% based on your own contribution.

What to submit, and when

<code>slides.pptx / .pdf</code>	Your presentation, in English.
<code>report.pdf</code>	6–10 pages: parts A, B and D written up. Must include plotted runtime charts, memory profiling screenshots, and the measurement tables..
<code>src/</code>	All C++ source, plus the ports to the other languages. Must include a Makefile or build command.
<code>data/</code>	The dataset, or the generator script that produces it.
<code>README.md</code>	How to build and run, expected output, machine and compiler used.
<code>grading/</code>	The evidence pack from Part F — added after your exercises are marked.

Deadlines

Week 1	Groups formed, topic confirmed
Week 2	One-page plan: demo problem + datasets
–7 days	Draft slides reviewed with the lecturer
–24 h	Final slides + code submitted
Day 0	Presentation; exercises published
+7 days	Class submissions due to your group
+14 days	Evidence pack + marks to the lecturer

RULES

Academic integrity & AI use

1

Cite everything

Book, chapter, page, or URL with access date. Code you adapted from elsewhere is cited in a comment at the point of use.

3

No shared code between groups

Discussing ideas is fine. Sharing source files is not. Both groups lose the marks.

5

Grade honestly

Peer marks are audited. Inflated or retaliatory marking is an integrity offence.

2

AI is a tool, not an author

You may use AI assistants. You must declare where, and you must be able to explain every line you submit, line by line, on request.

4

Measurements must be real

Fabricated timings are treated as fabricated data — zero for Parts C and D.

6

Everyone contributes

Submit a short contribution statement: who did what. Free riders are marked individually.

Self-check — tick all twelve

- | | |
|--|--|
| ☐ Every structure in our topic is placed in the syllabus, with section numbers. | ☐ We can say what problem each structure solves, and when not to use it. |
| ☐ The operation table is complete: best / average / worst / space, all justified. | ☐ The C++ demo compiles clean with -Wall on two different machines. |
| ☐ The demo solves a problem a real user would recognise. | ☐ The demo runs on $n \geq 100\,000$ and prints its own timings. |
| ☐ Edge cases are demonstrated, not just handled. | ☐ The same problem runs in Python, Java, JavaScript and Go, on our machine. |
| ☐ The comparison table is filled with our own numbers and our test conditions. | ☐ 3–5 exercises are written, with a marking scheme and our reference solutions. |
| ☐ Two markers are nominated, and the marking sheet template exists. | ☐ Every member has a speaking part and can answer questions on any part. |

If any box is unticked 24 hours before your slot, tell the lecturer — do not present unprepared.



START THIS WEEK

Form your group. Pick your topic. Find a real problem.



The demo is the hard part, and it is the part you cannot do the night before. Choose a problem you actually care about — you will be living with it all semester.

Questions?